



Backend

Guía completa del backend

En esta página construimos el backend de Dolt App paso a paso. Todo el código está en su versión final: usa `next` para delegar errores inesperados al manejador centralizado, limpia el avatar antiguo de Cloudinary al actualizar y tiene los handlers de error en `utils/`.

Objetivo

El backend debe permitir:

- registrar usuarios y hacer login
- subir avatar a Cloudinary (y eliminar el anterior al actualizarlo)
- obtener el perfil del usuario autenticado
- cambiar nombre de usuario, contraseña y avatar
- crear, editar, borrar y listar tareas
- filtrar tareas por estado o fecha y ordenar por fecha o estado

1. Crear el proyecto

```
npm init -y
```

```
npm install express mongoose dotenv cors bcrypt jsonwebtoken  
cloudinary multer multer-storage-cloudinary
```

`package.json` (editamos los scripts y declaramos el tipo de módulo):

```
{  
  "name": "backend",  
  "version": "0.0.1",
```

```
"main": "index.js",
"type": "commonjs",
"scripts": {
  "start": "node index.js",
  "dev": "node --watch index.js"
}
```

2. Estructura del proyecto

```
backend/
├── src/
│   ├── config/
│   │   ├── db.js
│   │   └── cloudinary.js
│   ├── controllers/
│   │   ├── users.controllers.js
│   │   └── tasks.controllers.js
│   ├── middlewares/
│   │   ├── auth.js
│   │   └── upload.js
│   ├── models/
│   │   ├── user.model.js
│   │   └── task.model.js
│   ├── routes/
│   │   ├── users.routes.js
│   │   └── tasks.routes.js
│   └── utils/
│       ├── errorHandler.js
│       └── notFoundHandler.js
├── .env
├── .gitignore
├── index.js
└── package.json
```

3. `.gitignore` y `.env`

```
node_modules
.env
```

```
PORT=3000
DB_URL=tu_url_de_mongodb
```

```
JWT_SECRET=tu_clave_secreta_jwt
CLOUDINARY_CLOUD_NAME=tu_cloud_name
CLOUDINARY_API_KEY=tu_api_key
CLOUDINARY_API_SECRET=tu_api_secret
FRONTEND_URL=http://localhost:5173
```

`FRONTEND_URL` permite cambiar el origen permitido por CORS sin tocar el código. En producción se cambia a la URL de Vercel directamente en Render.

4. `src/config/db.js`

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.DB_URL);
    console.log("Conectado con la base de datos");
  } catch (error) {
    // el servidor sigue arrancando aunque la BD falle; el
    // error aparece en consola
    console.log("Error conectando con la base de datos", error.message);
  }
};

module.exports = connectDB;
```

5. `src/config/cloudinary.js`

```
const cloudinary = require("cloudinary").v2;

// configuramos con las credenciales del .env
// NUNCA pongas estos valores directamente en el código
cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
  api_key: process.env.CLOUDINARY_API_KEY,
  api_secret: process.env.CLOUDINARY_API_SECRET,
});
```

```
// exportamos la instancia configurada para usarla en upload.js y en updateAvatar
module.exports = cloudinary;
```

6. `src/middlewares/upload.js`

```
const multer = require("multer");
const { CloudinaryStorage } = require("multer-storage-cloudinary");
const cloudinary = require("../config/cloudinary");

// CloudinaryStorage conecta multer con Cloudinary
// el archivo viaja directamente a la nube sin pasar por disco local
const storage = new CloudinaryStorage({
  cloudinary,
  params: {
    folder: "doitusers",
  },
  // carpeta en Cloudinary
  allowed_formats: ["jpg", "jpeg", "png", "webp"],
  // rechazamos otros formatos
},
});

const upload = multer({ storage });

// después de este middleware, req.file.path tiene la URL y req.file.filename el public_id
module.exports = upload;
```

7. `src/models/user.model.js`

```
const mongoose = require("mongoose");
const bcrypt = require("bcrypt");
```

```

const userSchema = new mongoose.Schema(
  {
    username: { type: String, required: true, trim: true, unique: true },
    email: {
      type: String,
      required: true,
      trim: true,
      unique: true,
      lowercase: true, // guardamos siempre en minúsculas
                        para evitar duplicados
    },
    password: {
      type: String,
      required: true,
      trim: true,
      minlength: [8, "La contraseña tiene que tener un mínimo de 8 caracteres"],
    },
    avatar: {
      type: String,
      default: "https://res.cloudinary.com/dwkafwila/image/upload/q_auto/f_auto/v1775587363/avatar_placeholder.png",
    },
  },
  { timestamps: true, versionKey: false }
);

// pre('save') se ejecuta antes de cada .save()
// usamos function() para tener acceso a 'this' (el documento que se va a guardar)
userSchema.pre("save", function () {
  // isModified evita re-cifrar si solo cambiamos username o avatar
  if (!this.isModified("password")) return;
  this.password = bcrypt.hashSync(this.password, 10);
});

```

```
const User = mongoose.model("User", userSchema);
module.exports = User;
```

8. `src/models/task.model.js`

```
const mongoose = require("mongoose");

const taskSchema = new mongoose.Schema(
  {
    title: { type: String, required: true, trim: true },
    date: { type: Date, required: true },
    status: {
      type: String,
      enum: ["pending", "completed"],
      default: "pending",
    },
    // referencia al usuario propietario
    // ref: "User" permite hacer populate() si en el futuro
    // necesitamos los datos del usuario
    user: { type: mongoose.Types.ObjectId, ref: "User", required: true },
  },
  { timestamps: true, versionKey: false }
);

const Task = mongoose.model("Task", taskSchema);
module.exports = Task;
```

9. `src/middlewares/auth.js`

```
const jwt = require("jsonwebtoken");
const User = require("../models/user.model");

// isAuth protege las rutas privadas
// si todo va bien llama a next() para que pase al controlador
```

```

// si algo falla responde con 401 y corta la cadena
const isAuth = async (req, res, next) => {
  try {
    const authorization = req.headers.authorization;

    if (!authorization) {
      return res.status(401).json({ message: "No se encuentra el token" });
    }

    // el token llega con el formato "Bearer <token>"; quitamos el prefijo
    const token = authorization.replace("Bearer ", "");

    // jwt.verify lanza excepción si el token ha expirado o es inválido
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    // comprobamos que el usuario sigue existiendo en BD
    // cubre el caso de que el usuario haya sido eliminado después de hacer login
    const user = await User.findById(decoded.id);
    if (!user) {
      return res.status(401).json({ message: "Token no válido" });
    }

    // guardamos el usuario en req para que los controladores no tengan que buscarlo de nuevo
    req.user = user;
    next();
  } catch (error) {
    // jwt.verify lanza aquí si el token está manipulado o ha caducado
    return res.status(401).json({ message: "Token no válido" });
  }
}

```

```
};  
  
module.exports = isAuth;
```

10. `src/utils/notFoundHandler.js`

```
// se ejecuta cuando ningún router coincide con la petición  
// solo 2 parámetros (req, res): no lleva next porque es un  
// endpoint final  
const notFoundHandler = (req, res) => {  
  res.status(404).json({ message: "Ruta no encontrada" });  
};  
  
module.exports = notFoundHandler;
```

`src/utils/errorHandler.js`

```
// Express lo reconoce como middleware de error porque tiene  
// exactamente 4 parámetros  
// se activa cuando un controlador llama a next(error)  
const errorHandler = (error, req, res, next) => {  
  console.error(error);  
  
  // si el error tiene statusCode propio lo usamos; si no,  
  // caemos en 500  
  const statusCode = error.statusCode || 500;  
  
  return res.status(statusCode).json({  
    message: error.message || "Error interno del servidor",  
  });  
};  
  
module.exports = errorHandler;
```

Por qué usar `next(error)` en los controladores

En vez de repetir `res.status(500).json(...)` en cada `catch`, delegamos el error inesperado al `errorHandler` con `next(error)`. Así si en algún momento queremos cambiar el formato del error 500, lo tocamos en un único sitio.

Los errores **esperados** (validación, credenciales, 404) siguen respondiendo directamente porque sabemos exactamente qué ha pasado.

11. `src/controllers/users.controllers.js`

```
const bcrypt = require("bcrypt");
const jwt = require("jsonwebtoken");
const User = require("../models/user.model");
const cloudinary = require("../config/cloudinary");

// — REGISTRO —

const register = async (req, res, next) => {
  try {
    const { username, email, password } = req.body;

    if (!username || !email || !password) {
      return res.status(400).json({ error: "Faltan campos obligatorios" });
    }

    const newUser = new User({
      username,
      email,
      password, // el pre('save') lo cifrará automáticamente
    });

    // req.file lo añade el middleware upload.single('avatar')
    // si no llega imagen, el modelo aplica el default (placeholder)
    const avatar = req.file ? req.file.path : undefined;
    newUser.avatar = avatar;

    await newUser.save();
  } catch (error) {
    next(error);
  }
};
```

```

    return res.status(200).json({ message: "Usuario creado"
});
  } catch (error) {
    // error de clave duplicada de MongoDB: email o usernam
e ya en uso
    // lo capturamos aquí para devolver { error } consisten
te con el resto de la API
    // si lo dejáramos caer a next(error), devolvería { mes
sage } y el frontend no lo detectaría
    if (error.code === 11000) {
      return res.status(400).json({ error: "El email o el u
sersname ya están en uso" });
    }
    next(error);
  }
};

// — LOGIN —————
const login = async (req, res, next) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({ error: "Faltan campos o
bligatorios" });
    }

    const user = await User.findOne({ email });
    if (!user) {
      // mismo mensaje para email y contraseña incorrectos:
no filtramos si el email existe
      return res.status(400).json({ error: "Credenciales in
correctas" });
    }

    const passwordMatch = bcrypt.compareSync(password, use
r.password);

```

```

    if (!passwordMatch) {
        return res.status(400).json({ error: "Credenciales incorrectas" });
    }

    // firmamos el token con solo el _id; caduca en 7 días
    const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, {
        expiresIn: "7d",
    });

    // devolvemos el usuario sin contraseña
    const userLogin = await User.findById(user._id).select("-password");
    return res.status(200).json({ token, user: userLogin });
} catch (error) {
    next(error);
}
};

// ———— OBTENER USUARIO AUTENTICADO ————
const getMe = async (req, res, next) => {
    try {
        // req.user lo añadió isAuth, pero aún tiene la contraseña
        // hacemos una nueva búsqueda para devolver solo los campos públicos
        const user = await User.findById(req.user._id).select("-password");
        return res.status(200).json(user);
    } catch (error) {
        next(error);
    }
};

// ———— ACTUALIZAR USERNAME ————

```

```

const updateUsername = async (req, res, next) => {
  try {
    const { username } = req.body;

    if (!username) {
      return res.status(400).json({ error: "Campo requerido" });
    }

    // modificamos req.user directamente (ya lo tenemos en memoria gracias a isAuthenticated)
    // el pre('save') no re-cifrará la contraseña porque no la hemos modificado
    req.user.username = username;
    await req.user.save();

    return res.status(200).json({ message: "Username modificado correctamente" });
  } catch (error) {
    next(error);
  }
};

// — ACTUALIZAR CONTRASEÑA —


---


const updatePassword = async (req, res, next) => {
  try {
    const { currentPassword, newPassword } = req.body;

    if (!currentPassword || !newPassword) {
      return res.status(400).json({ error: "Campos requeridos" });
    }

    const passwordMatch = bcrypt.compareSync(currentPassword, req.user.password);
    if (!passwordMatch) {

```

```

        return res.status(400).json({ error: "Credenciales incorrectas" });
    }

    // asignamos en texto plano; el pre('save') detectará que 'password' cambió
    // y la cifrará automáticamente antes de persistir
    req.user.password = newPassword;
    await req.user.save();

    return res.status(200).json({ message: "Contraseña modificada correctamente" });
  } catch (error) {
    next(error);
  }
};

// — ACTUALIZAR AVATAR —————
const updateAvatar = async (req, res, next) => {
  try {
    if (!req.file) {
      return res.status(400).json({ error: "Fichero requerido" });
    }

    // — Eliminar el avatar antiguo de Cloudinary —————

    // Solo lo hacemos si el avatar actual pertenece a la carpeta 'doitusers'
    // Así no intentamos borrar el placeholder (que no está en esa carpeta)
    const oldAvatarUrl = req.user.avatar;
    if (oldAvatarUrl && oldAvatarUrl.includes("/doitusers/")) {
      // Extraemos el public_id del formato: ../doitusers/<nombre_archivo>.<ext>
      // El public_id es lo que Cloudinary necesita para bo

```

```

    rrar: "doitusers/<nombre>"
    const match = oldAvatarUrl.match(/\/doitusers\/([^\./]+)\/);
    if (match) {
        const publicId = `doitusers/${match[1]}`;
        await cloudinary.uploader.destroy(publicId);
        // Si la destrucción falla no bloqueamos la actualización
        // el avatar huérfano se puede limpiar manualmente desde el panel de Cloudinary
    }
}

// — Guardar la nueva URL —————
// req.file.path contiene la URL de Cloudinary devuelta por CloudinaryStorage
req.user.avatar = req.file.path;
await req.user.save();

return res.status(200).json({ message: "Avatar modificado correctamente" });
} catch (error) {
    next(error);
}
};

module.exports = {
    register,
    login,
    getMe,
    updateUsername,
    updatePassword,
    updateAvatar,
};

```

Por qué importamos `cloudinary` aquí

`updateAvatar` necesita llamar a `cloudinary.uploader.destroy(publicId)` para borrar la imagen antigua. Por eso importamos la instancia ya configurada de `src/config/cloudinary.js`.

Cómo extraemos el `public_id`

Las URLs de Cloudinary para imágenes subidas a la carpeta `doitusers` tienen este formato:

```
https://res.cloudinary.com/cloud/image/upload/v123456789/doitusers/nombre.jpg
```

Con la regex `/\/doitusers\/([^.]+)/` extraemos el nombre del archivo sin extensión y reconstruimos el `public_id` como `doitusers/nombre`.

El placeholder (`avatar_placeholder.png`) no contiene `/doitusers/` en su URL, así que nunca entra en el bloque de borrado.

12. `src/controllers/tasks.controllers.js`

```
const Task = require("../models/task.model");

// ——— OBTENER TAREAS ———

const getTasks = async (req, res, next) => {
  try {
    const { status, date, sort } = req.query;

    // siempre filtramos por el usuario autenticado
    const filter = { user: req.user._id };

    // .includes() valida el valor antes de añadirlo al filtro de MongoDB
    if (status && ["pending", "completed"].includes(status)) {
      filter.status = status;
    }

    if (date) {
      const startDate = new Date(date);
      const endDate = new Date(date);
      endDate.setHours(23, 59, 59, 999); // cubrimos el día
```

```

a completo
    filter.date = { $gte: startDate, $lte: endDate };
  }

  // Task.find() prepara la consulta sin ejecutarla, para
  poder encadenar .sort()
  let query = Task.find(filter);

  if (sort === "date_asc")    query = query.sort({ date:
1  });
  if (sort === "date_desc")  query = query.sort({ date:
-1  });
  if (sort === "status_asc") query = query.sort({ statu
s: 1  });
  if (sort === "status_desc") query = query.sort({ statu
s: -1  });

  const tasks = await query;

  // devolvemos el array directamente; el frontend hace s
  etTasks(res || [])
  return res.status(200).json(tasks);
} catch (error) {
  next(error);
}
};

// — CREAR TAREA —————
const createTask = async (req, res, next) => {
  try {
    const { title, date, status } = req.body;

    if (!title || !date) {
      return res.status(400).json({ error: "El titulo y la
fecha son campos obligatorios" });
    }
  }

```

```

    const newTask = new Task({ title, date, status, user:
req.user._id });
    const taskSaved = await newTask.save();

    // devolvemos la tarea completa con su _id de MongoDB
    return res.status(200).json(taskSaved);
  } catch (error) {
    next(error);
  }
};

// — EDITAR TAREA —————
const updateTask = async (req, res, next) => {
  try {
    const { title, date, status } = req.body;

    // buscamos filtrando también por usuario: si la tarea
    es de otro, findOne devuelve null
    const task = await Task.findOne({ _id: req.params.id, u
ser: req.user._id });

    if (!task) {
      return res.status(404).json({ error: "No se encuentra
la tarea" });
    }

    // edición parcial: solo actualizamos los campos que ll
eguen
    if (title !== undefined) task.title = title;
    if (date !== undefined) task.date = date;
    if (status !== undefined) task.status = status;

    const taskUpdated = await task.save();
    return res.status(200).json(taskUpdated);
  } catch (error) {
    next(error);
  }
}

```

```

};

// — BORRAR TAREA —————

const deleteTask = async (req, res, next) => {
  try {
    // findOneAndDelete busca y borra en una sola operación
    atómica
    const task = await Task.findOneAndDelete({
      _id: req.params.id,
      user: req.user._id, // solo puede borrar sus propias
      tareas
    });

    if (!task) {
      return res.status(404).json({ error: "No se encuentra
      la tarea" });
    }

    return res.status(200).json({ message: "Tarea borrada c
    orrectamente" });
  } catch (error) {
    next(error);
  }
};

module.exports = { getTasks, createTask, updateTask, delete
Task };

```

13. src/routes/users.routes.js

```

const express = require("express");
const upload = require("../middlewares/upload");
const isAuth = require("../middlewares/auth");
const {
  register, login, getMe,
  updateUsername, updatePassword, updateAvatar,

```

```

} = require("../controllers/users.controllers");

const usersRoutes = express.Router();

// — Rutas públicas —
// upload.single('avatar') procesa el campo 'avatar' del fo
rm-data antes del controlador
usersRoutes.post("/register", upload.single("avatar"), regi
ster);
usersRoutes.post("/login", login);

// — Rutas privadas —
// isAuth verifica el token y añade req.user; siempre va AN
TES del controlador
usersRoutes.get("/me", isAuth, getMe);
usersRoutes.put("/username", isAuth, updateUsername);
usersRoutes.put("/password", isAuth, updatePassword);
usersRoutes.put("/avatar", isAuth, upload.single("avata
r"), updateAvatar);

module.exports = usersRoutes;

```

14. src/routes/tasks.routes.js

```

const express = require("express");
const isAuth = require("../middlewares/auth");
const { getTasks, createTask, updateTask, deleteTask } =
  require("../controllers/tasks.controllers");

const tasksRoutes = express.Router();

// todas las rutas de tareas son privadas
tasksRoutes.get("/", isAuth, getTasks); // GET /ap
i/tasks
tasksRoutes.post("/", isAuth, createTask); // POST /ap

```

```

i/tasks
tasksRoutes.put("/:id", isAuth, updateTask); // PUT /api/tasks/:id
tasksRoutes.delete("/:id", isAuth, deleteTask); // DELETE /api/tasks/:id

// /:id es un parámetro dinámico; llega en req.params.id de ntro del controlador

module.exports = tasksRoutes;

```

15. index.js

```

// dotenv debe cargarse primero para que el resto de módulos lean process.env correctamente
require("dotenv").config();

const express = require("express");
const cors = require("cors");
const connectDB = require("../src/config/db");
const usersRoutes = require("../src/routes/users.routes");
const tasksRoutes = require("../src/routes/tasks.routes");
const notFoundHandler = require("../src/utils/notFoundHandler");
const errorHandler = require("../src/utils/errorHandler");

const server = express();
const PORT = process.env.PORT || 3000;

connectDB();

// FRONTEND_URL viene del .env: "http://localhost:5173" en local,
// la URL de Vercel en producción – sin tocar el código

```

```

server.use(cors({
  origin: process.env.FRONTEND_URL || "http://localhost:5173",
}));

server.use(express.json());

server.use("/api/users", usersRoutes);
server.use("/api/tasks", tasksRoutes);

// el orden de los dos middlewares finales importa:
// primero el 404 (no tiene next, es un endpoint final)
// después el errorHandler (4 parámetros, Express lo reconoce como manejador de error)
server.use(notFoundHandler);
server.use(errorHandler);

server.listen(PORT, () => {
  console.log(`Servidor levantado en http://localhost:${PORT}`);
});

```

16. Cómo arrancar el backend

```
npm run dev
```

Deberías ver en consola:

- Conectado con la base de datos
- Servidor levantado en http://localhost:3000

17. Endpoints finales

Usuarios

POST	/api/users/register	→ crear cuenta (form-data con avatar opcional)
POST	/api/users/login	→ iniciar sesión (JSON)

GET	/api/users/me	→ perfil del usuario autenticado
PUT	/api/users/username	→ cambiar username (JSON)
PUT	/api/users/password	→ cambiar contraseña (JSON)
PUT	/api/users/avatar	→ cambiar avatar (form-data)

Tareas

GET	/api/tasks	→ listar tareas (array directo)
POST	/api/tasks	→ crear tarea
PUT	/api/tasks/:id	→ editar tarea (edición parcial)
DELETE	/api/tasks/:id	→ borrar tarea

Query params disponibles en **GET /api/tasks**

```
?status=pending|completed
?date=YYYY-MM-DD
?sort=date_asc|date_desc|status_asc|status_desc

# Combinados:
?status=pending&sort=date_asc
```

18. Cómo enviar los datos

Registro — **form-data**

username	→ texto
email	→ texto
password	→ texto
avatar	→ archivo (opcional; si no llega, el modelo pone el placeholder)

Login — **JSON**

```
{ "email": "correo@ejemplo.com", "password": "12345678" }
```

Crear tarea — **JSON**

```
{ "title": "Estudiar Node.js", "date": "2026-04-10", "status": "pending" }
```

Cambiar avatar — `form-data`

avatar → archivo de imagen

19. Tabla de respuestas

Tipo	Cuándo ocurre	Clave JSON
Éxito	operación realizada	<code>message</code> o el objeto creado
Error de validación (400)	campos vacíos, datos incorrectos	<code>error</code>
Recurso no encontrado (404)	tarea que no existe o no es tuya	<code>error</code>
Error de autenticación (401)	token ausente, inválido o caducado	<code>message</code>
Error inesperado (500)	fallo de BD, error de mongoose	<code>message</code> (vía <code>errorHandler</code>)

20. Pruebas en Insomnia

Asegúrate de que el backend está corriendo con `npm run dev` y de que MongoDB está conectado antes de empezar.

Configuración inicial

Crea una colección `DoIt App` con dos carpetas: `Users` y `Tasks`. Crea un entorno `Local`:

```
{
  "base_url": "http://localhost:3000",
  "token": ""
}
```

Después de hacer login, pega el token en `token` y usa `{{ token }}` en todas las peticiones privadas.

20.1 Registrar usuario

POST `http://localhost:3000/api/users/register` — Body: `Multipart Form`

```
username: mario
email:    mario@email.com
password: 12345678
avatar:   (archivo opcional)
```

Respuesta éxito: `{ "message": "Usuario creado" }`

Respuesta campos vacíos: `{ "error": "Faltan campos obligatorios" }`

Respuesta duplicado (400): `{ "error": "El email o el username ya están en uso" }`

20.2 Login

POST `http://localhost:3000/api/users/login` — Body: `JSON`

```
{ "email": "mario@email.com", "password": "12345678" }
```

Respuesta éxito: `{ "token": "...", "user": { "_id": "...", "username": "mario", "email": "...", "avatar": "..." } }`

Copia el `token` y pégalo en la variable de entorno de Insomnia.

20.3 Header para rutas privadas

```
Authorization: Bearer {{ token }}
```

Si falta o es inválido: `{ "message": "Token no valido" }` (401)

20.4 Obtener usuario (`GET /api/users/me`)

Respuesta: usuario sin campo `password`.

20.5 Cambiar username (`PUT /api/users/username`) — JSON

```
{ "username": "mario_nuevo" }
```

Respuesta: `{ "message": "Username modificado correctamente" }`

20.6 Cambiar contraseña (`PUT /api/users/password`) — JSON

```
{ "currentPassword": "12345678", "newPassword": "87654321"
}
```

Respuesta éxito: { "message": "Contraseña modificada correctamente" }

Contraseña incorrecta: { "error": "Credenciales incorrectas" }

20.7 Cambiar avatar (**PUT** /api/users/avatar) — Multipart Form

avatar: (archivo de imagen)

Respuesta: { "message": "Avatar modificado correctamente" }

El avatar antiguo se elimina automáticamente de Cloudinary si era de la carpeta `doitusers`.

Llama a `GET /api/users/me` después para verificar que `avatar` tiene la nueva URL.

20.8 Crear tarea (**POST** /api/tasks) — JSON

```
{ "title": "Hacer el proyecto final", "date": "2026-04-14",
  "status": "pending" }
```

Respuesta: la tarea completa con `_id`. Guarda ese `_id` para editar y borrar.

20.9 Obtener tareas (**GET** /api/tasks)

Respuesta: array directo. Si no hay tareas devuelve `[]`.

Prueba los filtros y la combinación:

```
GET /api/tasks?status=pending
GET /api/tasks?date=2026-04-14
GET /api/tasks?sort=date_desc
GET /api/tasks?status=pending&sort=date_asc
```

20.10 Editar tarea (**PUT** /api/tasks/:id) — JSON

```
{ "title": "Proyecto final corregido", "status": "completed" }
```

Respuesta: tarea actualizada. Solo envía los campos que quieras cambiar.

20.11 Borrar tarea (**DELETE** `/api/tasks/:id`)

Respuesta: `{ "message": "Tarea borrada correctamente" }`

20.12 Probar **notFoundHandler**

GET `http://localhost:3000/api/inventado` → `{ "message": "Ruta no encontrada" }` (404)

20.13 Orden recomendado de prueba

1. POST `/api/users/register`
2. POST `/api/users/login` → copiar token
3. GET `/api/users/me`
4. PUT `/api/users/username`
5. PUT `/api/users/password`
6. PUT `/api/users/avatar` → verificar borrado del antiguo en Cloudinary
7. GET `/api/users/me` → confirmar avatar y nombre actualizados
8. POST `/api/tasks` → crear varias tareas
9. GET `/api/tasks` → ver todas
10. GET `/api/tasks?status=pending`
11. GET `/api/tasks?sort=date_desc`
12. GET `/api/tasks?status=pending&sort=date_asc`
13. PUT `/api/tasks/:id`
14. DELETE `/api/tasks/:id`
15. GET `/api/inventado` → probar notFoundHandler